

## 1- Concepts de base du PHP Orienté Objet

### ❖ Qu'est-ce qu'un objet ?

Les **objets** pourraient être comparés à des **variables** dans lesquelles on pourra non seulement **écrire ou lire des valeurs**, mais également **effectuer des actions**.

#### Exemples de la vie quotidienne

Dans notre vie quotidienne, **nous utilisons des objets tous les jours**. Notre cafetière, notre voiture, notre téléviseur sont autant d'objets "quotidiens" qui pourraient se comparer à un objet PHP.

En effet, chacun de ces objets aura des **caractéristiques** qui lui sont propres, sa couleur, son énergie, sa taille, le nombre de places de la voiture. Chacune de ces caractéristiques vont définir notre **objet**.

Nous pourrions également effectuer des **actions** avec chacun de ces objets, démarrer, allumer, changer de chaîne, accélérer...

### ❖ Et le PHP dans tout ça ?

En PHP, les **caractéristiques** s'appelleront des **propriétés** et les **actions** s'appelleront des **méthodes**. Nos objets pourront posséder ces propriétés et ces méthodes en fonction de ce que nous souhaitons.

### ❖ Conventions

Avant d'aller plus loin, voici quelques **conventions importantes** dans le développement orienté objet.

- Chaque objet aura son propre fichier
- Le nom de fichier sera le nom de l'objet
- Le nom de l'objet sera écrit en **UpperCamelCase**

### ❖ Premier Classe

Une classe en PHP se définit par le mot clé "**class**".

Nous allons donc créer la Classe "**Compte**" dans un fichier intitulé "**Compte.php**" (notez-le C majuscule). Nous placerons ce fichier dans un dossier "**classes**" qui contiendront tous nos fichiers de classes.

Le fichier "**Compte.php**" contiendra donc la déclaration de classe "**Compte**" comme ceci

```
class Compte
{
}
```

Une fois la classe déclarée, à l'intérieur nous allons pouvoir déclarer les **propriétés de l'objet**. Notre compte pourra avoir un **titulaire** et un **solde**. Nous allons donc les déclarer comme ci-dessous (nous reviendrons sur le mot clé "public" dans la partie "Visibilité").

```
<?php
class Compte {
    public $titulaire;
    public $solde;
}
?>
```

```
<?php
/**
 * classe correspond à un compte bancaire
 */
class Compte {
    /**
     * Titulaire de compte
     * @var string
     */
    public $titulaire;

    /**
     * solde du compte
     *
     * @var float
     */
    public $solde;
}
?>
```

### ❖ Instancier l'objet

Après avoir déclaré la classe, nous pourrions l'appeler à plusieurs reprises dans des variables différentes. On appellera cette manipulation "**Instancier l'objet**" qui permet de créer plusieurs exemplaires de notre objet, chaque exemplaire étant appelé "**instance**".

```
require_once 'classes/Compte.php';
$compte1 = new Compte;
$compte2 = new Compte;

var_dump($compte1);
var_dump($compte1);
```

### ❖ Modifier la valeur des propriétés

Lorsque nous avons instancié l'objet "**Compte**", nous n'avons **pas défini de valeurs pour les propriétés "titulaire" et "solde"**.

Nous allons pouvoir les modifier comme ceci

```
require_once 'classes/Compte.php';
$compte1 = new Compte;
$compte2 = new Compte;

$compte1->titulaire = "Bentaleb";
$compte1->solde = 1000;

$compte2->titulaire = "Alaoui";
$compte2->solde = 500;

var_dump($compte1);
var_dump($compte2);
```

## POO en PHP

```
C:\wamp64\www\POO\index.php:25:
object(Compte)[1]
  public 'titulaire' => string 'Bentaleb' (length=8)
  public 'solde' => int 1000

C:\wamp64\www\POO\index.php:26:
object(Compte)[2]
  public 'titulaire' => string 'Alaoui' (length=6)
  public 'solde' => int 500
```

### ❖ Le constructeur

Comme nous l'avons vu précédemment, **lorsque nousinstancions notre classe, ses propriétés ne sont pas remplies.**

Nous allons pouvoir utiliser une méthode (fonction) spécifique appelée "**constructeur**" qui va nous permettre de "**construire**" notre objet lorsque nous l'instancions.

Cette méthode sera **exécutée automatiquement lors de l'instanciation** de l'objet.

```
<?php
/**
 * classe correspond à un compte bancaire
 */
class Compte {
    /**
     * Titulaire de compte
     * @var string
     */
    public $titulaire;

    /**
     * solde du compte
     * @var float
     */
    public $solde;

    public function __construct(string $titulaire,float $solde)
    {
        $this->titulaire = $titulaire;
        $this->solde = $solde;
    }
}
```

Vous remarquerez l'utilisation de "**this**" qui représente l'instance de classe que nous utilisons.

Pour instancier notre classe, nous devons donc maintenant **préciser un titulaire et un solde** de cette façon

```
require_once 'classes/Compte.php';
$compte1 = new Compte('bentaleb',1000);
$compte2 = new Compte('Alaoui',500);

var_dump($compte1);
var_dump($compte2);
```

### ❖ Ajouter des méthodes

Une fois notre objet créé, nous pouvons avoir besoin d'**effectuer des actions** sur celui-ci.

Dans le cas de notre "**Compte**", nous devrions pouvoir afficher le **solde**, **déposer** et **retirer** de l'argent.

Pour **afficher le solde du compte**, nous allons créer une méthode que nous placerons directement sous le constructeur.

Notre méthode affichera le solde du compte sous la forme d'un "**echo**" et sera créée comme ceci

```
public function voirSolde()
{
    echo "Le solde du compte est de $this->solde Dhs";
}
```

Pour afficher le solde de notre "**compte1**", nous allons donc utiliser la méthode de la façon suivante

```
$compte1->voirSolde();
```

## POO en PHP

```
C:\wamp64\www\POO\index.php:18:
object(Compte)[1]
  public 'titulaire' => string 'bentaleb' (length=8)
  public 'solde' => float 1000

C:\wamp64\www\POO\index.php:19:
object(Compte)[2]
  public 'titulaire' => string 'Alaoui' (length=6)
  public 'solde' => float 500

Le solde du compte est de 1000 Dhs
```

Pour pouvoir ajouter de l'argent à notre solde, nous allons créer une méthode "**deposer**"

```
public function depoter(float $montant)
{
    // On vérifie si le montant est positif
    if ($montant > 0) {
        $this->solde += $montant;
    }
}
```

Et l'utiliser comme ceci

```
$compte1->deposer(1000);
```

Pour le retrait, nous allons créer une méthode "**retirer**" qui vérifiera le solde avant d'autoriser le retrait

```
public function retirer(float $montant)
{
    // On vérifie le montant et le solde
    if ($montant > 0 && $this->solde >= $montant) {
        $this->solde -= $montant;
    } else {
        echo "Montant invalide ou solde insuffisant";
    }
}
```

Et l'utiliser comme ceci

```
$compte1->retirer(500);
```

## 2- Encapsulation (Visibilité)

Dans la 1ère partie nous avons vu comment créer un objet et comment l'instancier.

Nous avons utilisé un mot clé "**public**" pour déclarer nos propriétés et nos méthodes.

Ce mot clé permet de définir la **visibilité de nos propriétés et méthodes**, élément indispensable au bon fonctionnement de nos objets et de notre logique, basé sur le principe **d'encapsulation**.

Une bonne gestion de l'**encapsulation** permet de **protéger l'utilisation de votre objet** par les développeurs qui seraient amenés à en faire usage.

### ❖ Les différentes visibilités

Il existe 3 visibilités pour nos propriétés et méthodes :

- ✓ **public** : la propriété ou la méthode pourront être accédés depuis l'intérieur et l'extérieur de la classe
- ✓ **private** : l'accès à la propriété ou la méthode est possible uniquement depuis l'intérieur de la classe
- ✓ **protected** : équivalent à private mais accessible également dans les classes héritées

### ❖ Private et Protected

Nous allons traiter des visibilités "**private**" et "**protected**" simultanément, leur fonctionnement étant identique.

Si nous passons les propriétés "**titulaire**" et "**solde**" en private, il ne sera plus possible de les modifier ni d'y accéder directement.

Ainsi, si notre classe est définie comme ceci

```
/**
 * Classe correspondant à un compte bancaire
 */
class Compte
{
    /**
     * Titulaire du compte
     * @var string
     */
    private $titulaire;

    /**
     * Solde du compte
     * @var float
     */
    private $solde;
}
```

Nous ne pourrions pas accéder à nos propriétés de cette façon

```
echo $compte1->titulaire;
```

Nous aurons une erreur d'accès

| Fatal error: Uncaught Error: Cannot access private property Compte::\$titulaire in C:\wamp64\www\POO\index.php on line 22 |        |        |                |
|---------------------------------------------------------------------------------------------------------------------------|--------|--------|----------------|
| Error: Cannot access private property Compte::\$titulaire in C:\wamp64\www\POO\index.php on line 22                       |        |        |                |
| Call Stack                                                                                                                |        |        |                |
| #                                                                                                                         | Time   | Memory | Function       |
| 1                                                                                                                         | 0.0010 | 407064 | (main)()       |
|                                                                                                                           |        |        | ...index.php:0 |

## ❖ Les accesseurs

Afin de pouvoir définir et lire nos propriétés en "**private**", nous allons créer des "**accesseurs**", méthodes permettant d'y accéder.

### Les getters

La convention veut que la méthode s'appelle "**getPropriete**" où "**Propriete**" est à remplacer par le nom de la propriété concernée. Nous aurons donc "**getSolde**" et "**getTitulaire**" dans notre cas.

Les méthodes seront créées comme ceci

```
public function getTitulaire(): string
{
    return $this->titulaire;
}

public function getSolde(): float
{
    return $this->solde;
}
```

Nous pourrions donc accéder, par exemple, au titulaire du compte en écrivant

```
$compte1->getTitulaire();
```

### Les setters

Les "**setters**" permettent de définir la valeur des propriétés privées. Faire un "**setter**" permet, par exemple, de s'assurer que la valeur stockée dans la propriété est cohérente avec ce qui est prévu.

La convention veut que la méthode s'appelle "**setPropriete**" où "**Propriete**" est à remplacer par le nom de la propriété concernée. Nous aurons donc "**setSolde**" et "**setTitulaire**" dans notre cas.

Nous écrirons les "**setters**" de cette façon

```
public function setTitulaire(string $titulaire): self
{
    $this->titulaire = $titulaire;
    return $this;
}

public function setSolde(float $solde): self
{
    if ($solde >= 0) {
        $this->solde = $solde;
    }
    return $this;
}
```

Nous accéderons donc à notre "**setter**" de cette façon

```
// Ce code fonctionnera
$compte1->setSolde(100);
$compte1->setTitulaire('Bentaleb');

// Ce code provoquera une erreur
$compte1->setSolde('bentaleb');
```

## ❖ La méthode \_\_toString

La question ayant été posée lors du live, voici une description de la **méthode magique \_\_toString**.

Cette méthode permet de **définir ce qui sera renvoyé** par notre objet si quelqu'un essaie de l'**afficher au moyen d'un "echo"** par exemple.

Nous retournerons une chaîne de caractères de notre choix de la façon suivante

```
echo $compte1;
```

|  Catchable fatal error: Object of class Compte could not be converted to string in C:\wamp64\www\POO\index.php on line 49 |        |        |          |                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|--------|----------|----------------|
| Call Stack                                                                                                                                                                                                 |        |        |          |                |
| #                                                                                                                                                                                                          | Time   | Memory | Function | Location       |
| 1                                                                                                                                                                                                          | 0.0011 | 406600 | {main}() | ...index.php:0 |

```
public function __toString()
{
    return "Vous visualisez le compte de {$this->titulaire}, le solde est de {$this->solde} dhs";
}
```

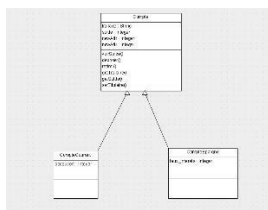
### 3- Héritages

#### Héritages

L'héritage en PHP permet de créer une nouvelle classe qui héritera des propriétés et des méthodes d'une classe parent et qui pourra, si on le souhaite, redéfinir certaines propriétés et méthodes.

Dans les diapositives précédentes, nous avons créé un objet "**Compte**" qui permet de **créer un compte bancaire**.

Il existe cependant plusieurs types de comptes bancaires, certains offrent une **possibilité de découvert**, d'autres permettent de **toucher des intérêts**.



Nous allons donc modifier notre objet "**Compte**" pour pouvoir gérer des "**sous-objets**" qui en hériteront.

Pour éviter d'instancier notre classe "**Compte**", nous allons la passer en classe abstraite par le mot clé "**abstract**".

Elle ne pourra donc plus être instanciée directement.

```
abstract class Compte
{
}
```

Une classe abstraite n'est pas « instanciable » et ne pourra donc pas « fabriquer » d'objets.

la classe Compte sera abstraite.

Elle servira de classe modèle pour celles qui en hériteront

De plus, toutes les propriétés qui pourront être manipulées dans le "**sous-objet**" devront être en visibilité "**public**" ou "**protected**". En effet, les propriétés en visibilité "**private**" ne sont pas disponibles par héritage.

```
/**
 * Titulaire du compte
 * @var string
 */
protected $titulaire;

/**
 * Solde du compte
 * @var float
 */
protected $solde;
```

Nous allons créer deux classes qui hériteront de la classe "**Compte**". Elles s'appelleront "**CompteCourant**" et "**CompteEpargne**" dans les fichiers du même nom.



```
class CompteCourant extends Compte
{
}
```

```
class CompteEpargne extends Compte
{
}
```

Nous devons, pour le moment, appeler chacun des fichiers avant de pouvoir les utiliser. Nous verrons dans l'article suivant une méthode pour charger automatiquement tous les fichiers nécessaires.

## Le compte courant

Focalisons-nous sur le **compte courant**.

C'est un compte bancaire dans lequel nous allons **autoriser un découvert**.

Son **constructeur** nécessitera **au moins les informations du compte bancaire** (titulaire et solde).

Le constructeur sera donc le suivant

```
class CompteCourant extends Compte
{
    private $decouvert;

    public function __construct(string $titulaire, float $solde, int $decouvert)
    {
        // On appelle le constructeur du parent
        parent::__construct($titulaire, $solde);

        // On définit les propriétés "locales"
        $this->decouvert = $decouvert;
    }
}
```

Nous pourrions donc **instancier un compte courant** de cette façon

```
// On instancie en précisant le découvert
$compte2 = new CompteCourant('Bentaleb', 2000, 200);
```

Etant donné que notre compte devra **bénéficier d'un découvert**, nous pourrions avoir **un solde négatif** jusqu'au montant du découvert.

Nous devons donc modifier la méthode "**retirer**" pour le "**CompteCourant**" spécifiquement.

Nous écrirons donc

```
public function retirer(float $montant)
{
    // On vérifie si le découvert permet le retrait
    if($this->solde - $montant > -$this->decouvert){
        $this->solde -= $montant;
    }else{
        echo 'Solde insuffisant';
    }
}
```

## **SURCHARGER UNE METHODE**

Appelé aussi « **override** ».

Lors d'un héritage, surcharger une méthode permet d'utiliser une fonction existante de la classe mère dans la classe fille tout en modifiant le comportement initialement prévu de celle-ci.

Enfin, la propriété "**decouvert**" étant privée, nous allons écrire ses accesseurs (Getter et Setter)

```

public function getDecouvert()
{
    return $this->decouvert;
}

public function setDecouvert(int $decouvert): self
{
    $this->decouvert = $decouvert;
    return $this;
}

```

## Le compte épargne

Pour le compte épargne, nous allons procéder de façon similaire afin de mettre en place le **versement des intérêts**.

Nous aurons donc une propriété "**taux\_interets**" qui correspondra au taux d'intérêts du compte et un **constructeur** en conséquence. Nous allons choisir de **rendre obligatoire la déclaration de ce taux** dans le constructeur.

```

class CompteEpargne extends Compte
{
    private $taux_interets;

    public function __construct(string $titulaire, float $solde, int $taux)
    {
        // On appelle le constructeur du parent
        parent::__construct($titulaire, $solde);

        // On définit les propriétés "locales"
        $this->taux_interets = $taux;
    }
}

```

Enfin, pour verser les intérêts, nous écrivons une méthode comme ceci

```

public function verserInterets(){
    if($this->solde > 0){
        $this->solde = $this->solde + ($this->solde * $this->taux_interets / 100);
    }else{
        echo 'Solde insuffisant';
    }
}

```

Sans oublier les accesseurs du taux d'intérêts

```

public function getTauxInterets()
{
    return $this->taux_interets;
}

public function setTauxInterets(int $taux): self
{
    $this->taux_interets = $taux;
    return $this;
}

```

## 4- Constantes de classe, attributs et méthodes statiques

Nous avons vu précédemment que les attributs et méthodes définis dans une classe appartiennent en fait à l'objet qui sera l'instance de cette classe. On s'est donc servi de l'opérateur `->` pour désigner un membre à partir de l'objet créé.

Cependant, on peut spécifier les membres qui appartiennent à la classe et non à l'objet. On appelle ces membres, **attributs et méthodes statiques**. Pour y accéder on se sert de l'opérateur de résolution statique `::` appelé simplement **double deux points**.

### Membres statiques d'une classe: constantes de classe, attributs et méthodes statiques

#### ❖ Constantes de classe

Une constante de classe est un élément statique par défaut. Son rôle est le même que celui d'une constante classique déclarée à l'aide de la fonction `define()`. Sa valeur est inchangée et elle appartient à la classe dans laquelle est-elle évoquée et non à l'objet qui constitue l'instance de classe.

Pour définir une constante on utilise le mot clé `const` suivi du nom de la constante à laquelle on affecte la valeur souhaitée. Par convention l'identifiant de la constante est déclaré en majuscule.

```
class Voiture
{
    const ROUES = 4;
    private $couleur = "Rouge";
    private $puissance;
    private $vitesse;
    public function accelerer()
    {
        echo "Appel de la méthode accelerer()";
    }
    public function ralentir()
    {
        echo "Appel de la méthode ralentir()";
    }
}
```

La constante de classe peut être appelée de l'intérieur comme de l'extérieur de la classe grâce à l'opérateur de résolution de portée `::`

```
<?php
| echo "Nombre de roues: ".Voiture::ROUES;
?>
```

#### ❖ Attributs et méthodes statiques

Les attributs et méthodes statiques appartiennent à la classe et non à l'objet. Par conséquent on ne peut pas y accéder par l'opérateur `->` mais plutôt par l'opérateur de résolution de portée `::` précédé par le nom de la classe dans laquelle ils sont définis.

Pour spécifier si un membre est statique on déclare le mot clé `static` après la visibilité. Par exemple:

```
class Voiture{
    private static $roues=4;

    public static function message(){
        echo "Il s'agit d'une voiture.<br />";
        echo "Elle a ".Voiture::$roues." roues.";
    }
}
```

```
<?php
| Voiture::message();
?>
```

Vous avez remarqué que nous avons appelé la méthode statique `message()` de l'extérieur de la classe, il est donc évident de déclarer le nom de la classe (`Voiture`) avant l'opérateur de résolution de portée. En revanche, l'attribut statique `$roues` a été appelé de l'intérieur de la classe. Là aussi on a mis le nom de

la classe avant l'opérateur de résolution de portée. Néanmoins, on aurait pu mettre le mot clé **self** qui désigne la classe courante.

L'exemple précédent peut donc devenir:

```
<?php
class Voiture{
    private static $roues=4;
    public static function message(){
        echo "Il s'agit d'une voiture.<br />";
        echo "Elle a ".self::$roues." roues.";
    }
}
Voiture::message();
```

Généralement, si on veut faire un appel statique (d'une constante, attribut ou méthode) à l'intérieur de la classe, il est préférable de mettre le mot clé **self** au lieu du nom de la classe elle même. Par contre, à l'extérieur de la classe il faut mettre le nom de celle ci avant l'opérateur de résolution de portée.

## 5- Méthodes magiques

Les méthodes magiques sont des outils très pratiques pour simplifier le code et automatiser certaines tâches.

Elles sont appelées automatiquement lorsque certaines actions sont effectuées et permettent d'éviter certaines erreurs.

**\_\_construct ()** : Constructeur de la classe ;

**\_\_destruct ()** : Destructeur de la classe ;

**\_\_set ()** : Déclenchée lors de l'accès en écriture à une propriété de l'objet ;

**\_\_get ()** : Déclenchée lors de l'accès en lecture à une propriété de l'objet ;

**\_\_call ()** : Déclenchée lors de l'appel d'une méthode inexistante de la classe (appel non statique) ;

**\_\_callstatic ()** : Déclenchée lors de l'appel d'une méthode inexistante de la classe (appel statique)

**\_\_isset ()** : Déclenchée si on applique isset () à une propriété de l'objet ;

**\_\_unset ()** : Déclenchée si on applique unset () à une propriété de l'objet ;

**\_\_sleep ()** : Exécutée si la fonction serialize () est appliquée à l'objet ;

**\_\_wakeup ()** : Exécutée si la fonction unserialize () est appliquée à l'objet ;

**\_\_toString ()** : Appelée lorsque l'on essaie d'afficher directement l'objet echo \$Object;

**\_\_set\_state ()** : Méthode statique lancée lorsque l'on applique la fonction var\_export () à l'objet ;

**\_\_clone ()** : Appelée lorsque l'on essaie de cloner l'objet ;

**\_\_autoload ()** : Cette fonction n'est pas une méthode, elle est déclarée dans le scope global et permet d'automatiser les "include/require" de classes PHP.

## 6- Les classes et méthodes abstraites : définition et intérêt

Une classe **abstraite** est une classe qui **ne peut pas être instanciée**.

Elle est **utilisée pour être héritée**.

Une **classe abstraite a au moins une opération abstraite**. Mais elle peut aussi avoir des opérations concrètes.

```
<?php
abstract class myClasse
{
    // C'est une fonction abstraite
    abstract function show();

    // Ce n'est pas une fonction abstraite
    function display()
    {
        echo "Hello World!";
    }
}
?>
```

### 1- Voici quelques règles importantes sur les classes abstraites en PHP.

```
<?php
// Classe abstraite
abstract class A {
    // fonction abstraite
    abstract function show();
}

class B extends A {
    function show() {
        echo "Hello World!";
    }
}

// La ligne suivante produit une erreur de compilation
// $a = new A();

$b = new B;
$b->show();
?>
```

Lors de l'héritage d'une classe abstraite (B → A), la méthode de la classe fille doit être définie avec le même nom. Ainsi, si la méthode abstraite est définie comme « protected », la méthode de la classe fille doit être définie comme « protected » ou « public », mais pas « private ». En plus, le type et le nombre d'arguments requis doivent être les mêmes. Pourtant, les classes fille peuvent avoir des arguments optionnels en plus.

### 2- Une classe abstraite ne peut pas contenir une méthode avec un corps. Si nous exécutons l'exemple suivant, alors il affichera un message d'erreur.

```
1. <?php
2.     abstract class A {
3.         abstract function show() {
4.             echo "Hello World!";
5.         }
6.     }
7. ?>
```

## 7- Les interfaces en PHP

Une interface permet aux utilisateurs de créer des programmes, en spécifiant les méthodes publiques qu'une classe doit implémenter, sans impliquer la complexité et les détails de l'implémentation des méthodes. Une interface est définie comme une classe, mais avec le mot-clé **interface**. L'interface ne contient pas de propriétés ou de variables comme le cas dans une classe.

### Exemple: Création d'une interface

L'exemple suivant définit une interface à l'aide du mot-clé **interface**.

```
1. <?php
2.     interface MyInterface {
3.         public function method1();
4.         public function method2();
5.         public function method3();
6.     }
7. ?>
```

Quelques points importants à retenir sur les interfaces en PHP :

- Une interface est constituée de méthodes qui n'ont pas d'implémentations, ce qui signifie que les méthodes d'interface sont des méthodes abstraites.
- Toutes les méthodes dans les interfaces doivent avoir une portée de visibilité public.
- Les interfaces sont différentes des classes car une classe peut hériter d'une seule classe alors que la classe peut implémenter une ou plusieurs interfaces.

Pour implémenter une interface, utilisez le mot-clé **implements** comme suit :

```
1. <?php
2. class MyClass implements MyInterface {
3.
4.     public function method1() {
5.
6.         // Implémentation de la méthode 1
7.     }
8.     public function method2(){
9.
10.        // Implémentation de la méthode 2
11.    }
12.    public function method3(){
13.
14.        // Implémentation de la méthode 3
15.    }
16.
17. }
18. ?>
```

### Exemple complète:

```
1. <?php
2.     interface MyInterface {
3.         public function method1();
4.         public function method2();
5.         public function method3();
6.     }
7.
8.     class MyClass implements MyInterface {
9.
10.        public function method1() {
11.            echo "Méthode1 Appelée". "\n";
12.        }
13.
14.        public function method2(){
15.            echo "Méthode2 Appelée". "\n";
16.        }
17.
18.        public function method3(){
19.            echo "Méthode3 Appelée". "\n";
20.        }
21.    }
22.
23.    $obj = new MyClass;
24.    $obj->method1();
25.    $obj->method2();
26.    $obj->method3();
27. ?>
```

### Implémenter plusieurs interfaces

Une classe peut implémenter plusieurs interfaces, séparées par des virgules.

```
1. class MyClass implements MyInterface1, MyInterface2, MyInterface3 {
2.     // code...
3. }
```

## 8- Les traits en PHP

PHP ne supporte que l'héritage simple, c'est à dire, une classe fille ne peut hériter que d'une seule classe mère. Donc, que faire si une classe a besoin d'hériter de plusieurs classes mères ? Les traits résolvent ce problème.

Les traits sont utilisés pour déclarer les méthodes qui peuvent être utilisées dans plusieurs classes. Les traits peuvent avoir des méthodes et des méthodes abstraites qui peuvent être utilisées dans plusieurs classes, et les méthodes peuvent avoir n'importe quel modificateur d'accès (public, private, or protected).

Les traits sont déclarés avec le mot-clé **trait** :

```
1. <?php
2.     trait myTrait {
3.         // code...
4.     }
5. ?>
```

Pour utiliser un trait dans une classe, utilisez le mot-clé **use** :

```
1. <?php
2.     class myClass {
3.         use myTrait;
4.     }
5. ?>
```

Regardons un exemple :

```
1. <?php
2.     trait school {
3.         public function learn() {
4.             echo "I am learning PHP at WayToLearnX!";
5.         }
6.     }
7.
8.     class Person {
9.         use school;
10.    }
11.
12.    $person = new Person();
13.    $person->learn();
14. ?>
```

Ici, nous déclarons un trait appelé « school ». Ensuite, nous créons une classe appelée « Person ». La classe utilise le trait, et toutes les méthodes du trait seront disponibles dans la classe.

Si d'autres classes ont besoin d'utiliser la fonction learn(), utilisez simplement le trait « school » dans ces classes. Cela réduit la duplication du code, car il n'est pas nécessaire de redéclarer la même méthode encore et encore.

### Utiliser plusieurs traits en PHP

```
1. <?php
2.     trait school {
3.         public function learn() {
4.             echo "I am learning PHP at WayToLearnX! \n";
5.         }
6.     }
7.
8.     trait company {
9.         public function work() {
10.            echo "I am working with PHP at WayToLearnX!";
11.        }
12.    }
13.
14.    class Person {
15.        use school, company;
16.    }
17.
18.    $person = new Person();
19.    $person->learn();
20.    $person->work();
21. ?>
```

Ici, nous avons déclaré deux traits appelés school et company. Ensuite, nous avons créé une classe appelée « Person ». La classe utilise les deux traits school et company qui sont séparés par une virgule.



## 9- Classe anonyme en PHP

Les classes anonymes sont des classes définies sans aucun nom. Elles ont été introduites dans PHP 7 pour permettre la création rapide des objets. Elles peuvent prendre des arguments via le constructeur, hériter d'autres classes, implémenter des interfaces, et utiliser des traits comme dans une classe normale.

En PHP 7, nous pouvons définir une classe anonyme en utilisant le mot-clé **new class**.

La syntaxe de base d'une classe anonyme est comme suite :

```
1. <?php
2. $obj1 = new class() {};
3.
4. $obj2 = new class($x, $y) {
5.     private $x;
6.     private $y;
7.     public function __construct($x, $y) {
8.         $this->x = $x;
9.         $this->y = $y;
10.    }
11. };
12. ?>
```

Nous utilisons le mot-clé « new », suivi du mot-clé « class », suivi des paramètres optionnels du constructeur, et enfin le corps de la classe qui est entre des accolades. Les objets instanciés par une classe anonyme ne sont pas différentes de celle des objets instanciés par une classe normale.

### Exemple complète:

```
1. <?php
2.
3. class MyClass { }
4.
5. interface MyInterface {}
6.
7. trait MyTrait {
8.     public function display() {
9.         return "Welcome to WayToLearnX.com";
10.    }
11. }
12.
13. $classX = new class extends MyClass implements MyInterface {
14.     use MyTrait;
15. };
16.
17. echo $classX->display();
18.
19. ?>
```

Comparées aux classes nommées, les classes anonymes sont à peu près pareils, Dans un sens où elles peuvent passer des paramètres au constructeur, hériter d'autres classes, implémenter des interfaces et utiliser des traits. Pourtant, les classes anonymes ne peuvent pas être sérialisées. Essayer de sérialiser une instance d'une classe anonyme, PHP génère une erreur fatale « Serialization of class@anonymous is not allowed... »

### Les classes finales et les méthodes finales en PHP

Le mot-clé **final** est utilisé en PHP pour les méthodes et les classes. Le mot-clé final pour les méthodes **empêche** la **surcharge** des **méthodes**, alors que le mot-clé **final** pour les **classes empêche l'héritage**.

#### Méthode finale

On ne peut pas surcharger la méthode de la classe mère.

```

1. <?php
2.     class MathClass{
3.         final function calculer($a, $b){
4.             $somme = $a + $b;
5.             echo "La somme de $a et $b est =" . $somme;
6.         }
7.     }
8.
9.     class School extends MathClass{
10.        function calculer($x, $y){
11.            $prod = $x * $y;
12.            echo "le produit de $x et $y est =" . $prod;
13.        }
14.    }
15.
16.    $obj= new School();
17.    $obj->calculer(2,4);
18. ?>

```

Dans l'exemple ci-dessus, la classe MathClass qui est la classe mère. Dans laquelle la méthode calculer est marquée comme finale. Cela implique que nous ne pouvons pas surcharger la méthode calculer dans aucune de ses classes filles. Pour identifier l'erreur, la classe fille School tente de définir la méthode calculer(). Cela produira une erreur fatale, cela implique que la méthode finale de la classe mère ne peut pas être définie dans sa classe fille.

### Classe finale

Une classe fille ne peut pas hériter une classe mère déclarée final.

```

1. <?php
2.     final Class A{
3.         function show(){
4.             echo "Welcome to WayToLearnX.com";
5.         }
6.     }
7.
8.     class B extends A{
9.         function show(){
10.            echo "Hello World!";
11.        }
12.    }
13.
14.    $b= new B();
15.    $b->show();
16. ?>

```

Dans l'exemple ci-dessus la classe A est définie avec le mot-clé final. Lorsque la classe B essaie d'hériter la classe mère A, PHP lève un message d'erreur qui dit : « La classe B ne peut pas hériter de la classe finale (A) », ce qui implique qu'il ne permet pas de créer une classe fille à partir de la classe mère.

## 10- Clonage d'objet en PHP

Les objets sont inconstants, ce qui signifie que l'objet a la capacité de changer l'état de l'information. Dans la [programmation PHP orientée objet POO](#), il peut arriver que vous ayez besoin de copies des objets. Nous pouvons créer une copie d'un objet en utilisant l'opérateur « = », pourquoi utiliser le clonage d'objet? Voici un exemple qui montre comment l'opérateur '=' fonctionne pour créer des copies d'un objet:

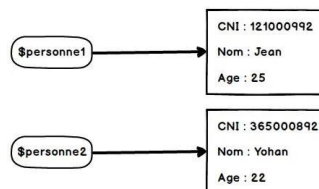
Commençons par créer une classe Personne simple.

```
1. <?php
2. class Personne{
3.     private $cni;
4.     private $nom;
5.     private $age;
6.
7.     public function __construct($cni,$nom,$age){
8.         $this->cni = $cni;
9.         $this->nom = $nom;
10.        $this->age = $age;
11.    }
12.
13.    public function __destruct(){
14.        echo sprintf("Personne avec le CNI: %s est détruit.",
15.            $this->cni);
16.    }
17.
18.    public function __toString(){
19.        return sprintf("CNI: %s, Nom: %s, Age: %s, ",
20.            $this->cni,
21.            $this->nom,
22.            $this->age);
23.    }
24. }
25.
26. }
27. ?>
```

Maintenant, nous pouvons créer des objets Personne:

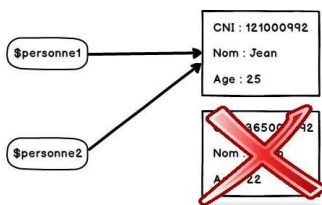
```
1. <?php
2. $personne1 = new Personne('121000992', 'Jean', '25');
3. echo $personne1;
4.
5. $personne2 = new Personne('365000892','Yohan','22');
6. echo $personne2;
7. ?>
```

Nous avons créé deux objets: \$personne1 et \$personne2. \$personne1 et \$personne2 sont des références qui pointent vers différents objets « Personne ».



Que se passera-t-il si nous faisons comme ça:

\$personne1 et \$personne2 référençant le même objet. Pourtant, le destructeur de \$personne2 est appelé car l'objet indiqué par \$personne2 n'est plus utilisé.



Comme nous l'avons vu, aucun objet réel n'a été créé lorsque nous avons affecté \$personne1 à \$personne2, seule la référence a été modifiée. La manière dont nous avons copié l'objet est appelée copie superficielle.

Pour effectuer une copie en profondeur entraînant la création d'un nouvel objet, nous utilisons l'opérateur **clone** comme suit:

```
1. $personne2 = clone $personne1;
```

Lorsque nous exécutons l'instruction ci-dessus, la référence \$personne2 pointe vers le nouvel objet créé. En plus, lors de l'exécution, la méthode magique **\_\_clone()** est invoquée automatiquement.

Nous pouvons ajouter la méthode magique **\_\_clone()** à la classe *Personne* pour voir comment cela fonctionne.

```
1. <?php
2.     public function __clone(){
3.         echo 'Copie un Objet <br>';
4.     }
5. ?>
```

## 11- Gestion des exceptions en PHP

Une exception est une erreur logique/système survenue lors de l'exécution normale d'un script. L'exception peut être levée par le système ou par le programme lui-même. Si l'exception ne peut pas être gérée, la fonction appelante doit informer le système.

### L'utilisation du bloc Try{...} Catch{...}

PHP5 introduit le bloc Try{...} Catch{...} pour intercepter les exceptions. Voir l'exemple ci-dessous.

```
1. <?php
2. try {
3.     division(1,0);
4. }
5. catch(Exception $e) {
6.     echo "Message : " . $e->getMessage();
7.     echo "Code : " . $e->getCode();
8. }
9. function division($a, $b) {
10.    if($b == 0) {
11.        throw new Exception("Erreur division par zéro", -1);
12.    }else{
13.        return $a/$b;
14.    }
15. }
16. ?>
```

Dans l'exemple ci-dessus, la méthode division() est appelée dans le bloc try{}. Le bloc try{} est la zone dans laquelle vous mettez votre code susceptible de générer une exception. En dessous du bloc try{} se trouve le bloc catch(). Le bloc catch attend un objet de type Exception en tant que paramètre. Dans le bloc catch(), vous mettez votre logique pour résoudre le problème ou l'erreur.

Dans la fonction division(), nous soulevons une exception à l'aide du mot clé « throw ». L'instruction qui suit « throw » est la syntaxe de création d'un nouvel objet de type Exception. La classe « Exception » accepte deux paramètres. Le premier paramètre est le message d'erreur et le deuxième paramètre est le code d'erreur que vous souhaitez affecter à cette erreur.

## 12- Les filtres en PHP

Les filtres PHP sont utilisés pour « Nettoyer » ou « assainir » et valider les données provenant des sources non sécurisées, comme les entrées des utilisateurs. Depuis PHP 5.2.0, les fonctions de filtrage sont activées par défaut. Vous pouvez les utiliser directement dans votre code sans aucune installation supplémentaire.

Pour valider les données en utilisant l'extension « filter », vous devez utiliser la fonction [filter\\_var\(\)](#) de PHP. La syntaxe de base de cette fonction est comme suite :

`filter_var(variable, filter, options)`

Cette fonction prend trois paramètres dont les deux derniers sont optionnels. Le premier paramètre est la valeur à filtrer, le deuxième paramètre est l'ID du filtre à appliquer, et le troisième paramètre est le tableau des options liées au filtre. Regardons comment cela fonctionne.

### Nettoyer une chaîne

L'exemple suivant va nettoyer une chaîne de caractères en supprimant toutes les balises HTML de celle-ci :

```
1. <?php
2. $str = "<p>Welcome <b>to</b> WayToLearnX.com</p>";
3.
4. // nettoyer la chaîne
5. $newStr = filter_var($str, FILTER_SANITIZE_STRING);
6. echo $newStr;
7. ?>
```

### Validation de valeurs entières

L'exemple suivant permettra de valider si la valeur est un entier valide ou non.

```
1. <?php
2. $nbr = 20;
3.
4. // Valider la valeur entière
5. if(filter_var($nbr, FILTER_VALIDATE_INT)){
6.     echo "$nbr est un entier valide";
7. } else{
8.     echo "$nbr n'est pas un entier valide";
9. }
10. ?>
```

Dans l'exemple ci-dessus, si la variable \$nbr est définie à 0, le code affichera « 0 n'est pas un entier valide ». Pour résoudre ce problème, vous devez tester explicitement la valeur 0, comme suit :

```
1. <?php
2. $nbr = 0;
3.
4. // Valider la valeur entière
5. if(filter_var($nbr, FILTER_VALIDATE_INT) === 0 || filter_var($nbr,
6. FILTER_VALIDATE_INT)){
7.     echo "$nbr est un entier valide";
8. } else{
9.     echo "$nbr n'est pas un entier valide";
10. }
11. ?>
```

### Valider les adresses IP

L'exemple suivant permettra de valider si la valeur est une adresse IP valide ou non.

```

1. <?php
2.     $ip = "192.168.20.1";
3.
4.     // Valider l'adresse IP
5.     if(filter_var($ip, FILTER_VALIDATE_IP)){
6.         echo "L'adresse IP est valide.";
7.     } else {
8.         echo "L'adresse IP n'est pas valide.";
9.     }
10. ?>

```

## Nettoyer et valider les adresses e-mails

L'exemple suivant vous montrera comment nettoyer et valider une adresse e-mail.

## Nettoyer et valider les URL

L'exemple suivant vous montrera comment nettoyer et valider une url.

```

1. <?php
2.     $url = "http://www.test.com";
3.
4.     // Supprimer tous les caractères illégaux dans l'URL
5.     $url = filter_var($url, FILTER_SANITIZE_URL);
6.
7.     // Valider l'URL
8.     if(filter_var($url, FILTER_VALIDATE_URL)){
9.         echo "L'URL est valide.";
10.    } else{
11.        echo "L'URL n'est pas valide.";
12.    }
13. ?>

```